

MAZE-SHOOTER

Generated by Doxygen 1.14.0

1 Class Index	1
1.1 Class List	1
2 File Index	3
2.1 File List	3
3 Class Documentation	5
3.1 Block Struct Reference	5
3.1.1 Detailed Description	5
3.1.2 Member Data Documentation	5
3.1.2.1 color	5
3.1.2.2 depth	5
3.1.2.3 height	6
3.1.2.4 isWall	6
3.1.2.5 pos	6
3.1.2.6 width	6
3.2 Coord Struct Reference	6
3.2.1 Detailed Description	6
3.2.2 Member Data Documentation	7
3.2.2.1 i	7
3.2.2.2 j	7
3.3 Element Struct Reference	7
3.4 Entity Struct Reference	7
3.4.1 Detailed Description	8
3.4.2 Member Data Documentation	8
3.4.2.1 ammo	8
3.4.2.2 maxAmmo	8
3.4.2.3 onGround	8
3.4.2.4 pitch	8
3.4.2.5 pos	8
3.4.2.6 size	8
3.4.2.7 type	9
3.4.2.8 velocityY	9
3.4.2.9 yaw	9
3.5 Projectile Struct Reference	9
3.5.1 Detailed Description	9
3.5.2 Member Data Documentation	9
3.5.2.1 active	9
3.5.2.2 life	10
3.5.2.3 owner	10
3.5.2.4 pos	10
3.5.2.5 radius	10
3.5.2.6 vel	10

3.6 SaveData Struct Reference	10
3.7 SaveFile Struct Reference	11
4 File Documentation	13
4.1 asset.h	13
4.2 bot.h	13
4.3 cryptage.h	13
4.4 level.h	14
4.5 menu.h	14
4.6 pile.h	14
4.7 player.h	14
4.8 projectile.h	14
4.9 sauvegarde.h	15
4.10 types.h	15
Index	17

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Block	Représente un bloc du labyrinthe (mur ou couloir)	5
Coord	Coordonnées pour la pile DFS (parcours du labyrinthe)	6
Element		7
Entity	Représente une entité du jeu (joueur ou bot)	7
Projectile	Représente un projectile (balle) dans le jeu	9
SaveData		10
SaveFile		11

Chapter 2

File Index

2.1 File List

Here is a list of all documented files with brief descriptions:

lib/headers/ asset.h	13
lib/headers/ bot.h	13
lib/headers/ cryptage.h	13
lib/headers/ level.h	14
lib/headers/ menu.h	14
lib/headers/ pile.h	14
lib/headers/ player.h	14
lib/headers/ projectile.h	14
lib/headers/ sauvegarde.h	15
lib/headers/ types.h	15

Chapter 3

Class Documentation

3.1 Block Struct Reference

Représente un bloc du labyrinthe (mur ou couloir).

```
#include <types.h>
```

Public Attributes

- Vector3 [pos](#)
- float [width](#)
- float [height](#)
- float [depth](#)
- Color [color](#)
- bool [isWall](#)

3.1.1 Detailed Description

Représente un bloc du labyrinthe (mur ou couloir).

3.1.2 Member Data Documentation

3.1.2.1 color

```
Color Block::color
```

Couleur du bloc pour le rendu.

3.1.2.2 depth

```
float Block::depth
```

Profondeur du bloc (axe Z).

3.1.2.3 height

```
float Block::height
```

Hauteur du bloc (axe Y).

3.1.2.4 isWall

```
bool Block::isWall
```

true si c'est un mur, false si c'est un couloir.

3.1.2.5 pos

```
Vector3 Block::pos
```

Position centrale du bloc.

3.1.2.6 width

```
float Block::width
```

Largeur du bloc (axe X).

The documentation for this struct was generated from the following file:

- lib/headers/types.h

3.2 Coord Struct Reference

Coordonnées pour la pile DFS (parcours du labyrinthe).

```
#include <types.h>
```

Public Attributes

- int [i](#)
- int [j](#)

3.2.1 Detailed Description

Coordonnées pour la pile DFS (parcours du labyrinthe).

3.2.2 Member Data Documentation

3.2.2.1 i

```
int Coord::i
```

Indice de ligne.

3.2.2.2 j

```
int Coord::j
```

Indice de colonne.

The documentation for this struct was generated from the following file:

- lib/headers/types.h

3.3 Element Struct Reference

Public Attributes

- [Coord](#) coord
- struct [Element](#) * next

The documentation for this struct was generated from the following file:

- src/pile.c

3.4 Entity Struct Reference

Représente une entité du jeu (joueur ou bot).

```
#include <types.h>
```

Public Attributes

- Vector3 [pos](#)
- float [yaw](#)
- float [pitch](#)
- float [velocityY](#)
- bool [onGround](#)
- float [size](#)
- int [ammo](#)
- int [maxAmmo](#)
- EntityType [type](#)

3.4.1 Detailed Description

Représente une entité du jeu (joueur ou bot).

3.4.2 Member Data Documentation

3.4.2.1 ammo

```
int Entity::ammo
```

Munitions actuelles.

3.4.2.2 maxAmmo

```
int Entity::maxAmmo
```

Capacité maximale du chargeur.

3.4.2.3 onGround

```
bool Entity::onGround
```

true si l'entité est au sol.

3.4.2.4 pitch

```
float Entity::pitch
```

Rotation verticale.

3.4.2.5 pos

```
Vector3 Entity::pos
```

Position de l'entité.

3.4.2.6 size

```
float Entity::size
```

Taille de l'entité.

3.4.2.7 type

```
EntityType Entity::type
```

Type de l'entité.

3.4.2.8 velocityY

```
float Entity::velocityY
```

Vitesse verticale.

3.4.2.9 yaw

```
float Entity::yaw
```

Rotation horizontale.

The documentation for this struct was generated from the following file:

- lib/headers/types.h

3.5 Projectile Struct Reference

Représente un projectile (balle) dans le jeu.

```
#include <types.h>
```

Public Attributes

- Vector3 [pos](#)
- Vector3 [vel](#)
- float [radius](#)
- bool [active](#)
- float [life](#)
- OwnerType [owner](#)

3.5.1 Detailed Description

Représente un projectile (balle) dans le jeu.

3.5.2 Member Data Documentation

3.5.2.1 active

```
bool Projectile::active
```

true si le projectile est actif.

3.5.2.2 life

```
float Projectile::life
```

Temps de vie restant (en secondes).

3.5.2.3 owner

```
OwnerType Projectile::owner
```

Propriétaire du projectile.

3.5.2.4 pos

```
Vector3 Projectile::pos
```

Position actuelle.

3.5.2.5 radius

```
float Projectile::radius
```

Rayon de la sphère (hitbox).

3.5.2.6 vel

```
Vector3 Projectile::vel
```

Vecteur vitesse (direction * vitesse).

The documentation for this struct was generated from the following file:

- lib/headers/types.h

3.6 SaveData Struct Reference

Public Attributes

- int **score**
- int **maxAmmo**
- float **x**
- float **y**
- float **z**
- float **yaw**
- float **pitch**
- int **onGround**
- int **ammo**

The documentation for this struct was generated from the following file:

- src/sauvegarde.c

3.7 SaveFile Struct Reference

Public Attributes

- [SaveData](#) **gameData**
- `uint32_t` **checksum**

The documentation for this struct was generated from the following file:

- `src/sauvegarde.c`

Chapter 4

File Documentation

4.1 asset.h

```
00001 #ifndef ASSET_H
00002 #define ASSET_H
00003
00004 #include "../linux/raylib-5.5_linux_amd64/include/raylib.h"
00005
00007 Texture2D ChargerTexture(const char *fileName);
00008
00010 void DessinerViseur(Texture2D texture, int screenWidth, int screenHeight);
00011
00013 void DessinerArme(Texture2D texture, int screenWidth, int screenHeight);
00014
00015 #endif
```

4.2 bot.h

```
00001 #ifndef BOT_H
00002 #define BOT_H
00003
00004 #include "types.h"
00005 #include "projectile.h"
00006 #include "../linux/raylib-5.5_linux_amd64/include/raymath.h"
00007
00009 void InitBot(Entity *bot);
00010
00012 void UpdateBot(Entity *bot, Block blocks[NUM_BLOCKS][NUM_BLOCKS], Vector3 targetPos, Projectile
    *projs); // Modifie l'état du bot et met à jour sa 'camera' + gère IA, physique et collisions
00013
00014 #endif
```

4.3 cryptage.h

```
00001 #ifndef CRYPTAGE_H
00002 #define CRYPTAGE_H
00003
00004 #include <string.h>
00005 #include <stdint.h>
00006
00008 void rc4_crypt(unsigned char *data, size_t data_len, const char *key, size_t key_len);
00009
00011 uint32_t calculate_checksum(const unsigned char *data, size_t len);
00012
00013 #endif
```

4.4 level.h

```
00001 #ifndef LEVEL_H
00002 #define LEVEL_H
00003
00004 #include "types.h"
00005
00007 void init_lab(Block blocks[NUM_BLOCKS][NUM_BLOCKS]);
00008
00010 void creer_lab(Block blocks[NUM_BLOCKS][NUM_BLOCKS]);
00011
00013 void DrawLevel(Block blocks[NUM_BLOCKS][NUM_BLOCKS]);
00014
00015 #endif
```

4.5 menu.h

```
00001 // menu.h
00002 #ifndef MENU_H
00003 #define MENU_H
00004
00005 #include "raylib.h"
00006 #include "types.h"
00007
00008 // Fonction pour dessiner un bouton et détecter un clic
00009 bool DessinerBouton(Rectangle rect, const char* texte);
00010
00011 // Fonction pour gérer la logique du menu
00012 void GererMenu(GameScreen* currentScreen);
00013
00014 #endif
```

4.6 pile.h

```
00001 #ifndef PILE_H
00002 #define PILE_H
00003
00004 #include "types.h"
00005
00007 void initpile(void);
00008
00010 int pilevide(void);
00011
00013 void empiler(int i, int j);
00014
00016 Coord depiler(void);
00017
00018 #endif
```

4.7 player.h

```
00001 #ifndef PLAYER_H
00002 #define PLAYER_H
00003
00004 #include "types.h"
00005 #include "../linux/raylib-5.5_linux_amd64/include/raymath.h"
00006
00008 void InitPlayer(Entity *player);
00009
00012 void UpdatePlayer(Entity *player, Block blocks[NUM_BLOCKS][NUM_BLOCKS], Camera3D *camera);
00013
00014 #endif
```

4.8 projectile.h

```
00001 #ifndef PROJECTILE_H
00002 #define PROJECTILE_H
00003
```

```

00004 #include "types.h"
00005 #include "../linux/raylib-5.5_linux_amd64/include/raymath.h"
00006 #include <stdio.h>
00007
00009 void InitProjectiles(Projectile *projs);
00010
00012 void ShootProjectile(Projectile *projs, Vector3 startPos, Vector3 direction, OwnerType owner);
00013
00015 void UpdateProjectiles(Projectile *projs, Block blocks[NUM_BLOCKS][NUM_BLOCKS], Entity *bot, Entity
    *player, int *score);
00016
00018 void DrawProjectiles(Projectile *projs);
00019
00020 #endif

```

4.9 sauvegarde.h

```

00001 #ifndef SAUVEGARDE_H
00002 #define SAUVEGARDE_H
00003
00004 #include <stdio.h>
00005 #include <stdlib.h>
00006 #include <string.h>
00007 #include "types.h"
00008
00011 void sauvegarder(Entity* player, int* score);
00012
00014 void chargerSauvegarde(Entity* player, int* score);
00015
00016 #endif

```

4.10 types.h

```

00001 #ifndef TYPES_H
00002 #define TYPES_H
00003
00004 #include "../linux/raylib-5.5_linux_amd64/include/raylib.h"
00005
00007 #define NUM_BLOCKS 101
00009 #define MAX_PROJ 50
00011 #define SCORE_TRADE 100
00012
00014 #define TRUE 1
00015 #define FALSE 0
00016
00021 // types.h
00022
00023
00024 typedef enum GameScreen { MENU, TEST, OPTIONS, EXIT } GameScreen; // Ajoute EXIT
00025
00026
00027
00032 typedef enum {
00033     ENTITY_PLAYER,
00034     ENTITY_BOT
00035 } EntityType;
00036
00041 typedef enum {
00042     OWNER_PLAYER,
00043     OWNER_BOT
00044 } OwnerType;
00045
00050 typedef struct {
00051     int i;
00052     int j;
00053 } Coord;
00054
00059 typedef struct {
00060     Vector3 pos;
00061     float width;
00062     float height;
00063     float depth;
00064     Color color;
00065     bool isWall;
00066 } Block;
00067
00072 typedef struct {
00073     Vector3 pos;

```

```
00074     Vector3 vel;
00075     float radius;
00076     bool active;
00077     float life;
00078     OwnerType owner;
00079 } Projectile;
00080
00081 typedef struct {
00082     Vector3 pos;
00083     float yaw;
00084     float pitch;
00085     float velocityY;
00086     bool onGround;
00087     float size;
00088     int ammo;
00089     int maxAmmo;
00090     EntityType type;
00091 } Entity;
00092 #endif
```

Index

active
 Projectile, 9

ammo
 Entity, 8

Block, 5
 color, 5
 depth, 5
 height, 5
 isWall, 6
 pos, 6
 width, 6

color
 Block, 5

Coord, 6
 i, 7
 j, 7

depth
 Block, 5

Element, 7

Entity, 7
 ammo, 8
 maxAmmo, 8
 onGround, 8
 pitch, 8
 pos, 8
 size, 8
 type, 8
 velocityY, 9
 yaw, 9

height
 Block, 5

i
 Coord, 7

isWall
 Block, 6

j
 Coord, 7

lib/headers/asset.h, 13
lib/headers/bot.h, 13
lib/headers/cryptage.h, 13
lib/headers/level.h, 14
lib/headers/menu.h, 14
lib/headers/pile.h, 14

lib/headers/player.h, 14
lib/headers/projectile.h, 14
lib/headers/sauvegarde.h, 15
lib/headers/types.h, 15

life
 Projectile, 9

maxAmmo
 Entity, 8

onGround
 Entity, 8

owner
 Projectile, 10

pitch
 Entity, 8

pos
 Block, 6
 Entity, 8
 Projectile, 10

Projectile, 9
 active, 9
 life, 9
 owner, 10
 pos, 10
 radius, 10
 vel, 10

radius
 Projectile, 10

SaveData, 10
SaveFile, 11

size
 Entity, 8

type
 Entity, 8

vel
 Projectile, 10

velocityY
 Entity, 9

width
 Block, 6

yaw
 Entity, 9